

# Übungsblatt 2: Scheduling-Theorie und kooperativer Scheduler

## Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

### Zielsetzung

In dieser Übung verknüpfen Sie die mathematische Scheduling-Theorie mit der praktischen Software-Entwicklung. Zunächst analysieren Sie ein Task-Set auf seine Planbarkeit mittels Rate Monotonic Scheduling (RMS). Anschließend implementieren Sie auf dem STM32H755 (Cortex-M4) Ihren eigenen kleinen kooperativen Round-Robin-Scheduler in C, um das Konzept des Task Control Blocks (TCB) und des Kontextwechsels zu verstehen. Abschließend messen Sie die Schwächen dieses Systems am Oszilloskop und provozieren einen System-Crash.

---

### Aufgabe 1: Scheduling-Theorie (RMS-Schranke)

Ein Steuergerät soll drei periodische Tasks verarbeiten. Für die Tasks wurden folgende Worst-Case Execution Times ( $C_i$ ) und Perioden ( $T_i$ ) ermittelt (Implicit Deadlines,  $D_i = T_i$ ):

- $\tau_1$ :  $C_1 = 1$  ms,  $T_1 = 4$  ms
- $\tau_2$ :  $C_2 = 2$  ms,  $T_2 = 6$  ms
- $\tau_3$ :  $C_3 = 1$  ms,  $T_3 = 12$  ms

1. Berechnen Sie die Gesamt-CPU-Auslastung  $U$ .
2. Vergeben Sie die Prioritäten der Tasks nach den Regeln des Rate Monotonic Scheduling (RMS).
3. Prüfen Sie mit der Liu-und-Layland-Schranke  $U \leq n(2^{1/n} - 1)$ , ob dieses System garantiert (hinreichend) schedulbar ist.

### Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

---

## Aufgabe 2: Die TCB-Datenstruktur anlegen

Wir bauen nun in der CubeIDE (Projekt auf dem Cortex-M4) einen simplen C-Scheduler. Definieren Sie in Ihrer `main.c` (im Bereich `USER CODE`) einen Task Control Block (TCB). Da wir noch keinen echten Assembler-Registertausch durchführen, benötigen wir keinen Stack-Pointer. Wir nutzen einen Funktionszeiger.

1. Erstellen Sie den `typedef` für einen Funktionszeiger `TaskFunction_t`.
2. Erstellen Sie das `struct MyTCB_t` mit den Variablen für den Funktionszeiger, den Zustand (`state`) und einer Schlaf-Zeit (`sleep_time`).
3. Legen Sie ein globales Array `task_list[3]` vom Typ `MyTCB_t` und einen Index `current_task_idx` an.

### Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

---

## Aufgabe 3: Die Scheduler-Logik (`os_yield`)

Implementieren Sie die Funktion `os_yield()`. Diese Funktion wird später von den Tasks freiwillig aufgerufen, um die CPU abzugeben (kooperatives Scheduling). Die Funktion soll ringförmig (Round-Robin) zum nächsten Index springen (modulo 3), prüfen, ob der Zustand der dort liegenden Task `READY` ist, und anschließend den Funktionszeiger der neuen Task ausführen.

**Hinweis (Didaktik):** Der direkte Aufruf von `task_func()` aus `os_yield()` ist eine bewusste Vereinfachung. Bei `while(1) + Yield` wächst der Call-Stack (kein echter Context Switch). In einem Produktions-OS kehrt man zum Dispatcher zurück.

### Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

---

## Aufgabe 4: Tasks ausführen & Jitter messen (Praxis)

Schreiben Sie drei kurze Funktionen `Task1()`, `Task2()` und `Task3()`.

1. Lassen Sie in `Task1()` einen GPIO-Pin auf `HIGH` und direkt vor dem Aufruf von `os_yield()` wieder auf `LOW` wechseln, um einen Messpunkt zu generieren. Die anderen Tasks können leere Schleifen-Durchläufe oder LED-Toggles beinhalten. Am Ende jeder Task muss `os_yield()` aufgerufen werden!
2. Füllen Sie das `task_list`-Array in der `main()` mit Ihren Funktionen, setzen Sie den Status auf `READY` und rufen Sie den ersten Funktionszeiger manuell auf, um das System zu starten.
3. Messen Sie den GPIO-Pin von `Task1()` am Oszilloskop. Was fällt Ihnen beim Timing auf?

## Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

---

## Aufgabe 5: Den Scheduler crashen

Wir zeigen nun die größte Schwachstelle von kooperativen Systemen auf.

1. Fügen Sie in `Task2()` eine lange Verzögerung ein (z. B. `HAL_Delay(1000)` oder eine endlose `while(1)`-Schleife). Verändern Sie nicht den Code in `Task1()` oder `Task3()`.
2. Flashen Sie das Programm und beobachten Sie das Oszilloskop und die LEDs.
3. Erklären Sie das beobachtete Verhalten und warum dies in echten Echtzeitsystemen inakzeptabel ist.

## Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*